Complete AWS CI/CD Deployment Guide

React + Spring Boot + MySQL on the Whizlabs AWS Sandbox

A Beginner-Friendly, GUI-Only, Step-by-Step Walkthrough

Verified against a live Whizlabs Sandbox deployment. Every step, field value, and setting in this guide has been tested and confirmed to work.

Table of Contents

---

1. What Are We Building?

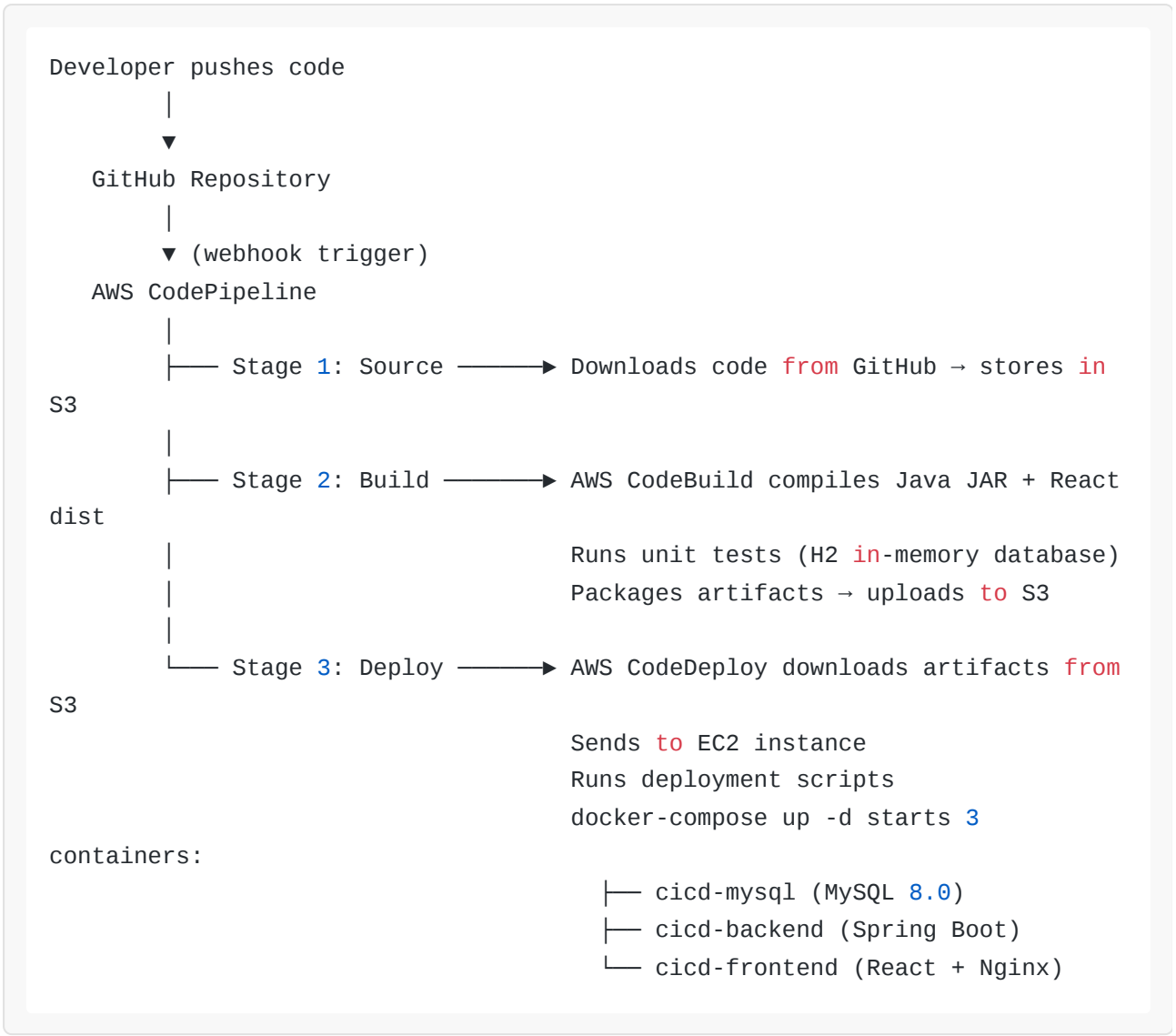
We are building a complete, production-style cloud deployment system for a full-stack web application. The system consists of three layers:

The Application is a web app that manages a list of items stored in a MySQL database. It has a React frontend (the user interface), a Spring Boot backend (the business logic and API), and a MySQL database (persistent storage). All three components run as Docker containers on a single EC2 virtual machine.

The Infrastructure is a set of AWS services that work together to host and run the application. These include an EC2 instance (the server), an S3 bucket (artifact storage), and IAM roles (permissions).

The CI/CD Pipeline is an automated system that watches your GitHub repository for new code commits. When you push a change, it automatically compiles the code, runs tests, and deploys the updated application to the server without any manual intervention.

The overall architecture is as follows:



---

2. Understanding the Application

What the Application Does

The application is an **Item Management System** — a simple CRUD (Create, Read, Update, Delete) application backed by a real MySQL database. It demonstrates a complete, realistic web application architecture.

The User Interface has three tabs:

Tab	Description
Items (MySQL RDS)	The main tab. Shows all items from the database with a search bar, status filter, and buttons to create, edit, and delete items.
Health	Shows the live health status of the backend API and how many items are in the database.
Pipeline Info	Explains the CI/CD architecture for educational purposes.

The REST API exposes the following endpoints:

Method	Endpoint	Description
`GET`	`/api/health`	Returns `{"status":"UP"}` with item count. Used by CodeDeploy to verify the deployment succeeded.
`GET`	`/api/items`	Returns all items from MySQL. Supports `?search=` and `?status=` query parameters.
`GET`	`/api/items/{id}`	Returns a single item by its database ID.
`POST`	`/api/items`	Creates a new item and saves it to MySQL.
`PUT`	`/api/items/{id}`	Updates an existing item in MySQL.
`DELETE`	`/api/items/{id}`	Deletes an item from MySQL.
`GET`	`/api/items/stats`	Returns total, active, and inactive item counts.

The Database is MySQL 8.0. On first startup, the application automatically creates the `items` table and seeds it with 8 sample items (AWS services like CodePipeline, CodeBuild, EC2, etc.).

---

3. Understanding the Codebase and File Structure

3.1 Complete File Tree

```
react-springboot-mysql-cicd/
│
├── buildspec.yml              ← AWS CodeBuild instructions
├── appspec.yml                ← AWS CodeDeploy instructions
├── docker-compose.yml         ← Multi-container Docker definition
├── .gitignore                 ← Files Git should ignore
│
├── backend/                   ← Spring Boot Java application
│   ├── Dockerfile             ← How to containerize the backend
│   ├── pom.xml                ← Maven build file (dependencies)
│   └── src/
│       ├── main/
│       │   ├── java/com/demo/cicd/
│       │   │   ├── CicdApplication.java        ← App entry point
│       │   │   ├── config/
│       │   │   │   └── DataInitializer.java    ← Seeds DB with sample data
│       │   │   ├── controller/
│       │   │   │   └── ApiController.java      ← REST endpoints
│       │   │   ├── model/
│       │   │   │   ├── Item.java               ← Database entity
│       │   │   │   └── HealthResponse.java     ← Health check response
│       │   │   ├── repository/
│       │   │   │   └── ItemRepository.java     ← Database queries
│       │   │   └── service/
│       │   │       └── ItemService.java        ← Business logic
│       │   └── resources/
│       │       └── application.properties      ← DB connection config
│       └── test/
│           ├── java/com/demo/cicd/
│           │   └── CicdApplicationTests.java   ← Unit tests
│           └── resources/
│               └── application.properties      ← H2 test config
│
├── frontend/                  ← React application
│   ├── Dockerfile             ← How to containerize the frontend
│   ├── nginx.conf             ← Nginx web server config
│   ├── index.html             ← HTML entry point
│   ├── package.json           ← Node.js dependencies
│   ├── vite.config.js         ← Vite build tool config
│   └── src/
```

```
│       ├── main.jsx           ← React bootstrap entry point
│       └── App.jsx            ← Main React component (full CRUD UI)
│
└── scripts/                   ← CodeDeploy lifecycle scripts
    ├── before_install.sh      ← Runs before files are copied
    ├── after_install.sh       ← Runs after files are copied
    ├── start_server.sh        ← Starts the Docker containers
    └── validate_service.sh    ← Verifies the app is running
```

3.2 Backend Files Explained

`CicdApplication.java` is the entry point of the Spring Boot application. It contains a single `main()` method that starts the embedded Tomcat web server. You never need to modify this file.

`Item.java` is the most important model file. It is annotated with `@Entity` and `@Table(name = "items")`, which tells Hibernate (the ORM layer) to map this Java class to a MySQL table called `items`. The fields map directly to database columns:

```java
@Entity
@Table(name = "items")
public class Item {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;           // AUTO_INCREMENT primary key

    private String name;       // Item name (required, max 100 chars)
    private String description;// Optional description (max 500 chars)
    private String status;     // "active" or "inactive"
    private LocalDateTime createdAt;  // Set automatically on insert
    private LocalDateTime updatedAt;  // Updated automatically on update
}
```

`ItemRepository.java` is an interface that extends Spring Data's `JpaRepository`. By extending this interface, you automatically get `save()`, `findById()`, `findAll()`, `deleteById()`, and `count()` methods without writing any SQL. The two custom methods use Spring Data's naming convention to generate queries automatically:

```
// Generates: SELECT * FROM items WHERE status = ? ORDER BY created_at DESC
List<Item> findByStatusOrderByCreatedAtDesc(String status);

// Generates: SELECT * FROM items WHERE LOWER(name) LIKE LOWER('%?%') ORDER
BY created_at DESC
List<Item> findByNameContainingIgnoreCaseOrderByCreatedAtDesc(String name);
```

`ItemService.java` contains the business logic layer. It is annotated with `@Service` and calls the repository. The service layer exists to separate business rules from the HTTP layer (the controller).

`ApiController.java` is the HTTP layer. It is annotated with `@RestController` and maps HTTP methods to Java methods. The `@CrossOrigin(origins = "*")` annotation allows the React frontend (running on a different port) to call the API. Key endpoints:

- `GET /api/items` — supports optional `?search=` and `?status=` query parameters
- `POST /api/items` — validates the request body with `@Valid` before saving
- `PUT /api/items/{id}` — returns 404 if the item does not exist
- `DELETE /api/items/{id}` — returns a confirmation message on success

`application.properties` configures the database connection. It uses environment variables with fallback defaults:

```
 # DB_HOST defaults to "mysql" — the Docker Compose service name
spring.datasource.url=jdbc:mysql://${DB_HOST:mysql}:${DB_PORT:3306}/${DB_NAME:cic

# JPA will automatically create/update the "items" table on startup
spring.jpa.hibernate.ddl-auto=update
```

`DataInitializer.java` runs once on startup. It checks if the `items` table is empty and, if so, inserts 8 sample items. This ensures the app has data to display on first deployment.

Test `application.properties` overrides the database configuration for unit tests, using H2 (an in-memory database) instead of MySQL. This means the tests run in AWS CodeBuild without needing a real database connection.

3.3 Frontend Files Explained

`App.jsx` is the entire React frontend in a single file. It is structured as follows:

- **`api` object**: A set of helper functions (`get` , `post` , `put` , `delete`) that wrap the browser's `fetch()` API. All requests go to `/api/...` , which Nginx proxies to the backend.
- **`ItemForm` component**: The create/edit form. It is reused for both creating new items and editing existing ones.
- **`ItemRow` component**: A single row in the items list, with Edit and Delete buttons.
- **`App` component**: The main component. It manages state (items, health, stats, search filter, status filter) and calls the API on load and after each CRUD operation.

`nginx.conf` is the Nginx configuration. The most important part is the `/api/` proxy block:

```
location /api/ {
    proxy_pass http://backend:8080/api/;
}
```

This tells Nginx: "When a browser requests `/api/items` , forward that request to the Spring Boot container (which is accessible at `http://backend:8080` within the Docker network)." The browser never talks directly to the backend — all traffic goes through Nginx on port 80.

3.4 CI/CD Configuration Files Explained

`buildspec.yml` is the instruction set for AWS CodeBuild. It has four phases:

Phase	What Happens
`install`	Downloads Maven dependencies and npm packages
`pre_build`	Runs Java unit tests using H2 in-memory database
`build`	Compiles the Spring Boot JAR and builds the React `dist/` folder
`post_build`	Copies the JAR, `dist/`, Dockerfiles, and scripts into a `deploy_artifacts/` folder

The `artifacts` section tells CodeBuild to upload everything inside `deploy_artifacts/` to S3.

`appspec.yml` is the instruction set for AWS CodeDeploy. It tells CodeDeploy to copy all files to `/home/ec2-user/app` on the EC2 instance, then run four lifecycle scripts in order:

```
BeforeInstall  → before_install.sh  (installs Docker if missing, stops old
containers)
AfterInstall   → after_install.sh   (sets file permissions, verifies Docker
is ready)
ApplicationStart → start_server.sh  (runs docker-compose up -d)
ValidateService  → validate_service.sh (checks /api/health returns "UP")
```

`docker-compose.yml` defines the three containers and how they connect:

```
cicd-mysql (port 3306) ──────────────────────────────────────────┐
                                                                 | app-
network
cicd-backend (port 8080) ─── depends on mysql being healthy ─────┤
(Docker bridge)
                                                                 |
cicd-frontend (port 80) ─── depends on backend being healthy ────┘
```

The `mysql_data` named volume ensures that even if the MySQL container is restarted or replaced, the data persists on the EC2 disk.

---

4. Understanding Dockerization

Docker allows us to package each component of the application (frontend, backend, database) into an isolated, self-contained unit called a **container**. Each container has everything it needs to run, regardless of what is installed on the host machine.

Why Pre-Built Artifacts?

A critical design decision in this project is that **CodeBuild does the compiling, not the EC2 instance**. Here is why:

A `t2.micro` EC2 instance has only 1 GB of RAM. Compiling a Spring Boot application with Maven requires downloading hundreds of MB of dependencies and running the Java compiler, which would exhaust the memory and cause the deployment to fail. Instead, CodeBuild (which has 3 GB of RAM by default) compiles the code and produces a ready-to-run `cicd-backend.jar` file. The EC2 instance only needs to run the JAR inside a lightweight container.

The Backend Dockerfile

```
 FROM eclipse-temurin:17-jre-alpine
# Use a minimal Java 17 runtime (not the full JDK)

RUN addgroup -S appgroup && adduser -S appuser -G appgroup
# Create a non-root user for security

COPY cicd-backend.jar app.jar
# Copy the pre-built JAR from CodeBuild artifacts

EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
# Run the JAR — Spring Boot starts its embedded Tomcat server
```

The Frontend Dockerfile

```
 FROM nginx:alpine
# Use a minimal Nginx web server image

COPY nginx.conf /etc/nginx/conf.d/default.conf
# Install our custom Nginx config (with the /api/ proxy)

COPY dist/ /usr/share/nginx/html
# Copy the pre-built React static files from CodeBuild artifacts

EXPOSE 80
# Nginx serves on port 80
```

---

5. How the CI/CD Pipeline Works

When you push code to GitHub, the following sequence of events occurs automatically:

Stage 1 — Source (approximately 10 seconds): CodePipeline detects the new commit via a GitHub webhook. It downloads the source code as a zip file and stores it in the S3 artifact bucket.

Stage 2 — Build (approximately 5–8 minutes): CodeBuild picks up the source zip from S3 and executes the `buildspec.yml` instructions. It installs Java 17 and Node.js 20, downloads Maven and npm dependencies, runs the Spring Boot unit tests (which use H2 so no database is needed), compiles the JAR, runs `npm run build` to produce the React `dist/` folder, and packages everything into a deployment zip. The zip is uploaded back to S3.

Stage 3 — Deploy (approximately 2–3 minutes): CodeDeploy downloads the deployment zip from S3 and sends it to the EC2 instance. The CodeDeploy agent on the EC2 instance runs the lifecycle scripts in order. The `start_server.sh` script runs `docker-compose up -d --build`, which starts the MySQL container first (and waits for it to be healthy), then starts the Spring Boot container (which connects to MySQL and creates/seeds the database), and finally starts the Nginx container. The `validate_service.sh` script polls the `/api/health` endpoint until it returns `{"status":"UP"}`, confirming the deployment succeeded.

---

6. Prerequisites

Before starting, ensure you have the following:

- A **Whizlabs AWS Sandbox** account with active credentials (Access Key ID and Secret Access Key).
- A **GitHub account** (free tier is sufficient).
- The **project zip file** (`aws-cicd-project.zip`) containing all the source code.
- A web browser (Chrome or Firefox recommended).

---

7. Part A — GitHub Setup

7.1 Create a GitHub Repository

1. Go to https://github.com and sign in.
2. Click the **+** icon in the top right corner and select **New repository**.
3. Fill in the form:
   - **Repository name**: `react-springboot-mysql-cicd`
   - **Visibility**: Public
   - **Do NOT** check “Add a README file”
4. Click **Create repository**.

7.2 Upload the Project Files

1. On the empty repository page, click **uploading an existing file**.
2. Extract the project zip file on your computer.
3. Drag and drop all the files and folders into the GitHub upload area.
4. Scroll down, enter a commit message such as `Initial project upload` , and click **Commit changes**.

7.3 Generate a Personal Access Token (PAT)

AWS CodePipeline needs a token to read your repository and set up a webhook.

1. Click your profile picture in the top right → **Settings**.
2. Scroll to the bottom of the left sidebar and click **Developer settings**.
3. Click **Personal access tokens** → **Tokens (classic)**.
4. Click **Generate new token** → **Generate new token (classic)**.
5. Fill in the form:
   - **Note**: `AWS CI/CD Pipeline`
   - **Expiration**: 90 days (or No expiration for a sandbox)
   - **Scopes**: Check **`repo`** (all sub-items) and **`admin:repo_hook`**
6. Click **Generate token**.
7. **Copy the token immediately** — it will not be shown again. Save it somewhere safe.

---

8. Part B — AWS Step 1: Create S3 Artifact Bucket

CodePipeline uses S3 to pass build artifacts between the Build and Deploy stages.

1. Log into the **AWS Console** at your Whizlabs sandbox URL.
2. Ensure the region selector in the top right shows **US East (N. Virginia) us-east-1**.
3. In the search bar at the top, type `S3` and click **S3**.
4. Click **Create bucket**.
5. Fill in the form:
   - **Bucket name**: `cicd-artifacts-XXXXXXXXXXXX` where `XXXXXXXXXXXX` is your 12-digit AWS Account ID. You can find your Account ID by clicking your username in the top right corner of the AWS Console.
   - **AWS Region**: `US East (N. Virginia) us-east-1`
   - Leave all other settings as default.
6. Scroll to the bottom and click **Create bucket**.

7. You should see a green success banner: “Successfully created bucket cicd-artifacts-XXXXXXXXXXXX”.

---

9. Part C — AWS Step 2: Create IAM Roles

IAM Roles grant AWS services permission to interact with each other. We need three roles.

Role 1: CodeBuildRole-CICD

This role allows AWS CodeBuild to read from S3, write build logs to CloudWatch, and access other CodeBuild features.

1. In the search bar, type `IAM` and click **IAM**.
2. In the left sidebar, click **Roles**.
3. Click **Create role**.
4. **Step 1 — Trusted entity type**: Select **AWS service**.
5. **Use case**: Scroll down and select **CodeBuild**, then click **Next**.
6. **Step 2 — Add permissions**: In the search box, type `AWSCodeBuildAdminAccess`. Check the box next to **AWSCodeBuildAdminAccess**. Click **Next**.
7. **Step 3 — Role name**: Enter `CodeBuildRole-CICD`.
8. Click **Create role**.

Role 2: CodeDeployRole-CICD

This role allows AWS CodeDeploy to read from S3 and interact with EC2 instances.

1. Click **Create role** again.
2. **Trusted entity type**: Select **AWS service**.
3. **Use case**: Scroll down and select **CodeDeploy**, then click **Next**.
4. **Add permissions**: The policy `AWSCodeDeployRole` should already be pre-selected. If not, search for it and check it. Click **Next**.
5. **Role name**: Enter `CodeDeployRole-CICD`.

6. Click **Create role**.

Role 3: CodePipelineRole-CICD

This role allows AWS CodePipeline to orchestrate the other services.

1. Click **Create role** again.
2. **Trusted entity type**: Select **AWS service**.
3. **Use case**: Scroll down and select **CodePipeline**, then click **Next**.
4. **Add permissions**: Search for `AWSCodeBuildAdminAccess` and check it. Click **Next**.
5. **Role name**: Enter `CodePipelineRole-CICD`.
6. Click **Create role**.

---

10. Part D — AWS Step 3: Launch EC2 Instance

This is the virtual server where your application will run.

10.1 Create a Key Pair

A key pair allows you to SSH into the EC2 instance if needed.

1. In the search bar, type `EC2` and click **EC2**.
2. In the left sidebar, under **Network & Security**, click **Key Pairs**.
3. Click **Create key pair**.
4. Fill in the form:
   - **Name**: `cicd-keypair`
   - **Key pair type**: RSA
   - **Private key file format**: `.pem`
5. Click **Create key pair**. The `.pem` file will download to your computer. Keep it safe.

10.2 Create a Security Group

A security group acts as a firewall, controlling what traffic can reach the EC2 instance.

1. In the left sidebar, under **Network & Security**, click **Security Groups**.
2. Click **Create security group**.
3. Fill in the form:
    - **Security group name**: `cicd-sg`
    - **Description**: `Allow HTTP and SSH for CI/CD demo`
4. Under **Inbound rules**, click **Add rule** twice to add these two rules:

Type	Protocol	Port Range	Source
SSH	TCP	22	Anywhere-IPv4 (0.0.0.0/0)
HTTP	TCP	80	Anywhere-IPv4 (0.0.0.0/0)

1. Click **Create security group**.

10.3 Launch the Instance

1. In the left sidebar, click **Instances**.
2. Click **Launch instances**.
3. Fill in the form:
    - **Name**: `react-springboot-app`
    - **Application and OS Images**: Select **Amazon Linux 2023 AMI** (it should be the first option under “Quick Start”).
    - **Instance type**: `t2.micro` (the free tier eligible option).
    - **Key pair**: Select `cicd-keypair` from the dropdown.
    - **Firewall (security groups)**: Select **Select existing security group** and choose `cicd-sg`.
4. Scroll down to **Advanced details** and expand it.
5. Scroll to the very bottom of the Advanced details section to find the **User data** text area.

6. Paste the following script **exactly as written** into the User data field:

```
#!/bin/bash
yum update -y
yum install -y ruby wget
cd /home/ec2-user
wget https://aws-codedeploy-us-east-1.s3.us-east-
1.amazonaws.com/latest/install
chmod +x ./install
./install auto
systemctl enable codedeploy-agent
systemctl start codedeploy-agent
yum install -y docker
systemctl enable docker
systemctl start docker
usermod -aG docker ec2-user
curl -SL https://github.com/docker/compose/releases/download/v2.23.0/docker-
compose-linux-x86_64 -o /usr/local/bin/docker-compose
chmod +x /usr/local/bin/docker-compose
```

1. Scroll to the bottom and click **Launch instance**.
2. You will see a success message. Click **View all instances**.
3. Wait for the **Instance State** to change from `Pending` to `Running` (this takes about 1–2 minutes). Note down the **Public IPv4 address** — you will need it later.

> ***Important**: The User data script runs in the background after the instance starts. It installs Docker, Docker Compose, and the CodeDeploy agent. This takes approximately 3–5 minutes. Do not proceed to the next steps until at least 5 minutes have passed after the instance reaches the `Running` state.*

---

11. Part E — AWS Step 4: Create CodeBuild Project

CodeBuild is the service that compiles your code.

1. In the search bar, type `CodeBuild` and click **CodeBuild**.
2. Click **Create build project**.
3. **Project configuration**:

- **Project name**: `react-springboot-build`

4. **Source**:
   - **Source provider**: GitHub
   - Click **Connect using OAuth** or **Connect with a GitHub personal access token**.
   - If using a token, paste the PAT you generated in Part A.
   - **Repository**: In my GitHub account → Select `react-springboot-mysql-cicd`.
5. **Primary source webhook events**: Leave unchecked (CodePipeline will trigger builds).
6. **Environment**:
   - **Environment image**: Managed image
   - **Compute**: EC2
   - **Operating system**: Ubuntu
   - **Runtime(s)**: Standard
   - **Image**: `aws/codebuild/standard:7.0`
   - **Image version**: Always use the latest image for this runtime version
   - **Service role**: Existing service role → `CodeBuildRole-CICD`
7. **Buildspec**:
   - Select **Use a buildspec file** (CodeBuild will look for `buildspec.yml` in the root of the repository).
8. **Artifacts**: Leave as **No artifacts** (CodePipeline manages artifact storage).
9. **Logs**:
   - Check **CloudWatch logs** so you can see build output if something goes wrong.
10. Click **Create build project**.

---

12. Part F — AWS Step 5: Create CodeDeploy Application

CodeDeploy is the service that deploys the compiled code to the EC2 instance.

12.1 Create the Application

1. In the search bar, type `CodeDeploy` and click **CodeDeploy**.
2. In the left sidebar, click **Applications**.
3. Click **Create application**.
4. Fill in the form:
   - **Application name**: `react-springboot-app`
   - **Compute platform**: EC2/On-premises
5. Click **Create application**.

12.2 Create the Deployment Group

1. Inside the application you just created, click **Create deployment group**.
2. Fill in the form:
   - **Deployment group name**: `react-springboot-dg`
   - **Service role**: Select `CodeDeployRole-CICD` from the dropdown.
   - **Deployment type**: In-place
3. **Environment configuration**:
   - Select **On-premises instances** (this is the critical Whizlabs sandbox workaround — do NOT select "Amazon EC2 instances").
   - Under the tag group, set:
     - **Key**: `Name`
     - **Value**: `react-springboot-app`
4. **Deployment settings**: Leave as `CodeDeployDefault.AllAtOnce`.
5. **Load balancer**: **Uncheck** "Enable load balancing".
6. Click **Create deployment group**.

---

13. Part G — AWS Step 6: Create CodePipeline

CodePipeline is the orchestrator that connects GitHub, CodeBuild, and CodeDeploy into a single automated workflow.

1. In the search bar, type `CodePipeline` and click **CodePipeline**.
2. Click **Create pipeline**.
3. **Step 1 — Pipeline settings**:
   - **Pipeline name**: `react-springboot-pipeline`
   - **Execution mode**: Superseded
   - **Service role**: Existing service role → `CodePipelineRole-CICD`
   - **Advanced settings** → **Artifact store**: Custom location → Select your `cicd-artifacts-XXXXXXXXXXXX` bucket.
4. Click **Next**.
5. **Step 2 — Source stage**:
   - **Source provider**: GitHub (Version 1)
   - Click **Connect to GitHub** and authorize AWS to access your GitHub account.
   - **Repository**: `react-springboot-mysql-cicd`
   - **Branch**: `main`
   - **Change detection options**: GitHub webhooks (recommended)
6. Click **Next**.
7. **Step 3 — Build stage**:
   - **Build provider**: AWS CodeBuild
   - **Region**: US East (N. Virginia)
   - **Project name**: `react-springboot-build`
8. Click **Next**.
9. **Step 4 — Deploy stage**:
   - **Deploy provider**: AWS CodeDeploy
   - **Region**: US East (N. Virginia)
   - **Application name**: `react-springboot-app`

- **Deployment group**: `react-springboot-dg`

10. Click **Next**, review all settings, and click **Create pipeline**.

The pipeline will start running immediately. The **Source** stage will succeed quickly. The **Build** stage will take 5–8 minutes. The **Deploy** stage will **fail** at this point — this is expected because we have not yet configured the CodeDeploy agent on the EC2 instance. Proceed to Part H.

---

14. Part H — Step 7: The Whizlabs Sandbox Workaround

Why This Step Is Necessary

In a standard AWS environment, you would attach an IAM Instance Profile to the EC2 instance. This gives the CodeDeploy agent on the instance a set of AWS credentials automatically via the EC2 metadata service. However, the Whizlabs sandbox IAM policy blocks the `iam:CreateInstanceProfile` action, so this standard approach does not work.

The solution is to register the EC2 instance as an **On-Premises Instance** in CodeDeploy and provide the credentials directly in the CodeDeploy agent configuration file. This is a fully supported AWS feature designed for deploying to servers outside of AWS.

14.1 Register the Instance in CodeDeploy (via CloudShell)

1. Click the **CloudShell** icon (`>_`) in the top navigation bar of the AWS Console. A terminal will open at the bottom of the screen.

2. First, get your exact IAM user ARN by running:

   ```
   aws sts get-caller-identity
   ```

   Note the value of the `"Arn"` field. It will look like: `arn:aws:iam::012433734806:user/Whiz_User_339680.67051941`

3. Register the EC2 instance as an on-premises instance (replace `YOUR_ARN` with the ARN from the previous command):

```
aws deploy register-on-premises-instance \
 --instance-name react-springboot-app \
 --iam-user-arn YOUR_ARN
```

4. Add the tag so the Deployment Group can find this instance:

```
aws deploy add-tags-to-on-premises-instances \
 --instance-names react-springboot-app \
 --tags Key=Name,Value=react-springboot-app
```

5. Verify the registration was successful:

```
aws deploy list-on-premises-instances
```

   You should see `react-springboot-app` in the output.

14.2 Configure the CodeDeploy Agent on EC2 (via Instance Connect)

1. Go to the **EC2** console and click **Instances**.
2. Select your `react-springboot-app` instance.
3. Click the **Connect** button at the top.
4. Select the **EC2 Instance Connect** tab and click **Connect**. A browser-based terminal will open.
5. In the terminal, switch to the root user:

```
sudo su
```

6. Create the on-premises configuration file. This is the **exact format** required by the CodeDeploy agent. Replace the placeholder values with your actual Whizlabs

sandbox credentials:

```
cat > /etc/codedeploy-agent/conf/codedeploy.onpremises.yml <<
'ENDOFFILE'
---
aws_access_key_id: YOUR_ACCESS_KEY_ID
aws_secret_access_key: YOUR_SECRET_ACCESS_KEY
iam_user_arn: YOUR_IAM_USER_ARN
region: us-east-1
ENDOFFILE
```

> ***Critical Note**: When using `iam_user_arn`, the AWS credentials (`aws_access_key_id` and `aws_secret_access_key`) **must be placed directly inside this file**. A separate credentials file will not work with this configuration mode.*

1. Update the main CodeDeploy agent configuration to point to the on-premises file:

```
echo ":on_premises_config_file: /etc/codedeploy-
agent/conf/codedeploy.onpremises.yml" >> /etc/codedeploy-
agent/conf/codedeployagent.yml
```

2. Restart the CodeDeploy agent to apply the new configuration:

```
systemctl restart codedeploy-agent
```

3. Verify the agent is running and polling successfully:

```
tail -20 /var/log/aws/codedeploy-agent/codedeploy-agent.log
```

   You should see lines containing `"HTTP_STATUS":"200"` and `poll_host_command`. This confirms the agent is authenticated and waiting for deployment commands.

---

15. Part I — Step 8: Trigger the Pipeline and Verify

15.1 Trigger the Pipeline

1. Go to the **CodePipeline** console and click on `react-springboot-pipeline`.
2. Click the **Release change** button in the top right.
3. Click **Release** in the confirmation dialog.

15.2 Monitor the Pipeline

Watch the three stages progress:

Stage	Expected Duration	What to Watch For
Source	~10 seconds	Status changes to "Succeeded"
Build	5–8 minutes	Click "Details" to see the CodeBuild log in real time
Deploy	2–4 minutes	Status changes to "Succeeded"

If the Build stage fails, click **Details** to open the CodeBuild log. Look for the error message in the `BUILD` or `POST_BUILD` phase.

If the Deploy stage fails, go to **CodeDeploy** → **Deployments** → click the latest deployment ID → click the instance ID to see which lifecycle event failed and the error message.

15.3 Verify the Live Application

1. Go to the **EC2** console and copy the **Public IPv4 address** of your instance (e.g., `54.234.136.202`).
2. Open a new browser tab and navigate to `http://YOUR_EC2_IP` (use `http://`, not `https://`).
3. You should see the **AWS CI/CD Pipeline Demo** application with 8 pre-seeded items.
4. Test the database by clicking **+ New Item**, filling in a name and description, and clicking **Create Item**. The "Total in RDS" counter should increase.

5. Test the API directly by visiting `http://YOUR_EC2_IP/api/health` in your browser. You should see:

```
{"status":"UP","service":"cicd-backend","database":"MySQL (AWS RDS)","dbItemCount":9}
```

---

16. Part J — Testing the CI/CD Loop

The most powerful feature of a CI/CD pipeline is that it deploys automatically when you push new code. Test this now using GitHub's built-in web editor.

1. Go to your `react-springboot-mysql-cicd` repository on GitHub.
2. Navigate to `frontend/src/App.jsx`.
3. Click the **pencil icon** (Edit this file) in the top right.
4. Find the line that says `React + Spring Boot +` and change the text to include today's date or any other small change.
5. Scroll down and click **Commit changes**.
6. Immediately go back to the **CodePipeline** console. Within 30 seconds, you should see the pipeline start running automatically — triggered by the GitHub webhook.
7. Wait for all three stages to complete, then refresh your browser tab showing the live application. You should see your change.

Congratulations! You have successfully built and tested a complete, automated CI/CD pipeline.

---

17. Lab Submission Requirements

To prove that you have successfully completed this lab, you must submit the following evidence. Please gather these screenshots and text outputs as you complete the final verification steps.

Evidence 1: The Live Application UI

Take a screenshot of the live application running in your browser.

- **What to show**: The main “Items (MySQL RDS)” tab showing the pre-seeded items, plus at least one new item you created yourself.
- **Why**: Proves the frontend is deployed, the backend is responding, and the MySQL database is persisting data.
- **Requirement**: The browser address bar showing the EC2 instance’s Public IP address **must** be visible in the screenshot.

Evidence 2: The Health API Response

Open a new browser tab and navigate to `http://YOUR_EC2_IP/api/health`.

- **What to show**: A screenshot of the JSON response.
- **Why**: Proves the Spring Boot backend is healthy and successfully connected to the MySQL database.
- **Requirement**: The response must show `"status":"UP"` and `"database":"MySQL (AWS RDS)"`.

Evidence 3: The CodePipeline Success State

Take a screenshot of the AWS CodePipeline console.

- **What to show**: The `react-springboot-pipeline` details page showing all three stages (Source, Build, Deploy).
- **Why**: Proves the CI/CD automation was successfully configured and executed.
- **Requirement**: All three stages must show a green **Succeeded** status.

Evidence 4: The CI/CD Loop Verification

Take a screenshot of your GitHub commit history OR the live application showing your custom change from Part J.

- **What to show**: The custom text you added to `App.jsx` visible on the live website, OR the GitHub commit showing the change.

- **Why**: Proves the webhook is working and the pipeline automatically deploys new code.

Submission Format

Paste all four screenshots into a single Word document or PDF, label them clearly (Evidence 1, Evidence 2, etc.), and submit the document to your instructor or learning management system.

---

18. Troubleshooting Reference

Symptom	Likely Cause	Solution
Deploy stage fails with "No instances found"	The on-premises instance is not registered or the tag does not match	Re-run the CloudShell commands in Step 7.1 and verify the tag `Name=react-springboot-app` is set
Deploy stage fails with "AccessDeniedException"	The CodeDeploy agent is not using the on-premises config	SSH into EC2 and verify `/etc/codedeploy-agent/conf/codedeploy.onpremises.yml` exists with the correct credentials
Deploy stage fails with "Missing credentials"	The `onpremises.yml` file is missing the `aws_access_key_id` field	The credentials must be inline in `onpremises.yml`, not in a separate file
Build stage fails with "Connection reset"	Transient Maven network error in CodeBuild	Click "Release change" in CodePipeline to retry the pipeline
App shows "OFFLINE" in the header	Spring Boot container has not started yet	Wait 2–3 minutes and refresh. MySQL takes ~30 seconds to initialize before Spring Boot can connect.
App is accessible but items list is empty	DataInitializer did not run	Check the Spring Boot logs: `docker logs cicd-backend` via EC2 Instance Connect

---

19. Glossary

Term	Definition
CI/CD	Continuous Integration / Continuous Deployment. An automated practice of building, testing, and deploying code on every commit.
CodePipeline	The AWS service that orchestrates the CI/CD workflow by connecting source, build, and deploy stages.
CodeBuild	The AWS service that compiles source code, runs tests, and produces deployable artifacts.
CodeDeploy	The AWS service that copies artifacts to EC2 instances and runs deployment scripts.
S3	Amazon Simple Storage Service. Used here to store build artifacts between pipeline stages.
IAM Role	An AWS identity with specific permissions that can be assumed by AWS services.
EC2	Amazon Elastic Compute Cloud. A virtual server in the cloud.
Docker	A platform for packaging applications into portable, isolated containers.
Docker Compose	A tool for defining and running multi-container Docker applications using a YAML file.
Container	A lightweight, isolated unit that packages an application and all its dependencies.
Image	A read-only template used to create Docker containers.
Nginx	A high-performance web server used here to serve the React static files and proxy API requests.
Spring Boot	A Java framework for building REST APIs and web applications with minimal configuration.
JPA / Hibernate	Java Persistence API and its implementation. Maps Java objects to database tables automatically.
On-Premises Instance	A CodeDeploy feature for deploying to servers that are not EC2 instances (or EC2 instances without an IAM instance profile).

Term	Definition
UserData	A script that runs automatically when an EC2 instance first starts. Used here to install Docker and the CodeDeploy agent.
Artifact	A compiled, packaged, deployable file produced by the build stage (e.g., a JAR file or a zip of static files).
Webhook	An HTTP callback that GitHub sends to CodePipeline when new code is pushed, triggering the pipeline automatically.